

Algorithm Design Project (Phase 1): Prompt Optimization

Algorithm-Project-PromptOptimization-Group11

February 13, 2026

Abstract

This report presents Phase 1 of a prompt optimization project: formal problem definition, analysis, algorithm design, pseudocode, complexity analysis, handcrafted examples, initial testing design, and a clean GitHub repository skeleton. We treat prompt optimization as a discrete search/optimization problem where candidate prompts are generated via explicit edit operators, evaluated using known metrics on QA and summarization datasets, and optimized using classic algorithms (Hill Climbing, BFS, Beam Search) with heuristic constraints. An LLM backend is modular: `MockLLM` provides deterministic unit tests, while a local `Ollama` runtime enables cost-free real evaluations.

1 Problem Definition

1.1 Goal

Given a task dataset D , a prompt generation mechanism G , an evaluator E , and a search budget B , find a prompt p maximizing a known score:

$$p = \arg \max_{p \in \mathcal{P}} m(p),$$

where $\mathcal{P}$ is the prompt search space reachable from an initial prompt p_0 via operators in G .

1.2 Inputs

- Task type $T \in \{\text{qa}, \text{summarization}\}$
- Dataset $D = \{(x_i, y_i)\}_{i=1}^n$ (input and reference output)
- Initial prompt p_0
- Prompt generator G (edit operators defining neighbors)
- LLM backend M (local Ollama model or MockLLM for tests)
- Metric(s): Exact Match (QA) and ROUGE-L (summarization)
- Budget B : maximum number of prompt evaluations

1.3 Outputs

- Best prompt p
- Best score $m(p)$
- Search trace (CSV): evaluated prompts, scores, best-so-far (for convergence plot)

1.4 Simple vs. Complex Versions

Simple version (Phase 1): single objective metric, small fixed datasets, deterministic evaluation (temperature 0) or MockLLM.

Complex version (Phase 2): multi-objective (quality vs cost/latency), larger datasets (≥ 10 real-world inputs), stochastic outputs, memory/time profiling, algorithm variants (simulated annealing, genetic algorithm).

2 Problem Analysis

2.1 Category

Prompt optimization is a **discrete optimization** problem over a structured space (strings with semantics) and can be modeled as:

- **Graph search:** nodes are prompts, edges are edit operators.
- **Local optimization:** hill climbing over a neighbor function.
- **Heuristic search:** constrained branching using pruning rules.

2.2 Search Space Growth and Hardness Intuition

The number of reachable prompts grows rapidly with:

- the number of constraints available,
- available output formats and styles,
- inclusion of verification or example blocks.

This yields combinatorial growth similar to classic NP-hard selection problems (choose/arrange instruction fragments). In practice, exhaustive search is infeasible under limited evaluation budgets, so we rely on approximations and heuristics.

2.3 LLM Role in the Pipeline

We designed the system so LLM usage is explicit and modular:

- **Response generation (required for real evaluation):** the LLM produces $\hat{y}_i = M(p, x_i)$.
- **Candidate generation (optional):** in Phase 1 we use rule-based edit operators; Phase 2 can add LLM-suggested edits.
- **Quality checking:** deterministic known metrics (Exact Match / ROUGE-L). Phase 2 may add LLM-as-judge.
- **Similarity/pruning:** Phase 1 uses string-based novelty (Jaccard). Phase 2 can use embeddings.

3 Metrics and Evaluation

3.1 QA Metric: Exact Match

For QA, we compute Exact Match (EM) after normalization (lowercasing, punctuation and whitespace normalization). EM is 1 if prediction matches reference, else 0.

3.2 Summarization Metric: ROUGE-L

For summarization, we compute token-level ROUGE-L F1 using the LCS (Longest Common Subsequence) between prediction and reference.

3.3 Scoring Function

We define:

$$m(p) = q(p) - \lambda \cdot \text{cost}(p),$$

where $q(p)$ is average metric score on dataset D . In Phase 1, λ can be set to 0 for pure quality comparisons, while we still log cost proxies for future work.

4 Search Space and Prompt Generation

4.1 Prompt Representation

Prompts are represented as structured objects:

- `task, instruction, constraints[], output_format, style, verification.`

They are deterministically rendered into text for hashing and reproducibility.

4.2 Edit Operators (Neighbor Function)

We implement rule-based operators such as:

- Add/Remove/Swap constraint
- Set output format (`plain/json/bullets`)
- Change style (`concise/formal/step_by_step`)
- Enable/disable verification step
- Reorder constraints

Neighbor generation is capped by `max_neighbors` to control branching factor.

5 Algorithm Design and Pseudocode

All algorithms share the same components:

1. Generate candidate prompts via neighbor function G (optionally constrained by heuristics).
2. Evaluate prompts using LLM backend + metrics.
3. Select best prompt under a known metric and evaluation budget.

5.1 Hill Climbing (Primary)

Algorithm 1 Hill Climbing for Prompt Optimization

```
1: procedure HILLCLIMB( $p_0, E, D, B, I$ )
2:    $p \leftarrow p_0$ 
3:    $(s, \_) \leftarrow E(p, D)$ 
4:    $best \leftarrow p, bestScore \leftarrow s$ 
5:   for  $t = 1$  to  $I$  do
6:     if evaluations  $\geq B$  then
7:       break
8:     end if
9:      $N \leftarrow \text{Neighbors}(p)$ 
10:     $p' \leftarrow p, s' \leftarrow s$ 
11:    for all  $q \in N$  do
12:      if evaluations  $\geq B$  then
13:        break
14:      end if
15:       $(sq, \_) \leftarrow E(q, D)$ 
16:      if  $sq > s'$  then
17:         $p' \leftarrow q, s' \leftarrow sq$ 
18:      end if
19:    end for
20:    if  $s' \leq s$  then
21:      break
22:    end if
23:     $p \leftarrow p', s \leftarrow s'$ 
24:    if  $s > bestScore$  then
25:       $best \leftarrow p, bestScore \leftarrow s$ 
26:    end if
27:  end for
28:  return ( $best, bestScore$ )
29: end procedure
```

5.2 BFS on Prompt Space

BFS explores prompts level-by-level up to depth L , with a visited set and evaluation budget.

5.3 Beam Search

Beam Search keeps the top- k prompts at each depth and expands them, trading completeness for efficiency.

5.4 Heuristic-Constrained Search

We implement heuristic filtering:

- Length cap (max prompt words)

- Novelty pruning via Jaccard similarity threshold

This reduces the effective branching factor and speeds up search.

6 Handcrafted Examples

6.1 QA Example (Format Sensitivity)

Dataset item: *Capital of France?* reference = *Paris*.

Prompt A: “Answer the question.” may yield “The answer is Paris.” which fails EM.

Prompt B: Add constraint “Answer with only the final answer (no extra text).” yields “Paris” and passes EM.

Thus, the optimization landscape strongly depends on instruction formatting constraints.

6.2 Summarization Example (Constraint Sensitivity)

Given a multi-sentence paragraph, a prompt with constraint “Keep the summary under 3 sentences” can reduce verbosity and improve overlap with concise references, increasing ROUGE-L.

7 Complexity Analysis

Let:

- b = branching factor (neighbors per prompt, capped by `max_neighbors`)
- L = search depth
- k = beam width
- I = hill climbing iterations
- n = dataset size
- C_M = cost of one model generation per example

Evaluation of one prompt costs $O(n \cdot C_M)$.

7.1 Hill Climbing

Prompt evaluations $\leq I \cdot b$ (budgeted), total time:

$$O(I \cdot b \cdot n \cdot C_M).$$

Space: $O(1)$ plus trace logs and cache.

7.2 BFS

Worst-case nodes to depth L : $\sum_{i=0}^L b^i = O(b^L)$. Time:

$$O(b^L \cdot n \cdot C_M),$$

Space:

$$O(b^L)$$

(visited + queue), hence depth/budget constraints are critical.

7.3 Beam Search

Evaluations $\approx k \cdot b \cdot L$ (budgeted). Time:

$$O(k \cdot b \cdot L \cdot n \cdot C_M),$$

Space: $O(k \cdot b)$.

7.4 Best/Worst/Average Cases

Hill climbing best-case stops immediately (local optimum); worst-case evaluates many neighbors per iteration. BFS worst-case is exponential. Beam provides a tunable trade-off.

8 Repository Structure and Phase-1 Deliverables

8.1 Repository Layout

```
src/  
  prompt/ (representation , operators)  
  evaluation/ (metrics , evaluator , MockLLM, OllamaLLM)  
  algorithms/ (hill_climbing , bfs , beam_search , heuristics)  
  utils/ (hashing , trace , datasets)  
tests/ (unit and smoke tests)  
scripts/ (run_hill_climb , run_search)  
reports/ (phase1.tex)  
results/ (.gitkeep; trace CSVs ignored)
```

8.2 Branching and Commits

Phase 1 is developed on branch **phase-1** and merged to **main** via pull request. Commits are staged and meaningful (e.g., “implement beam search for prompt optimization”).

8.3 Initial Tests (Phase 1)

Unit tests validate:

- deterministic prompt rendering and neighbor generation,
- metric correctness (Exact Match, ROUGE-L),
- evaluator scoring pipeline using MockLLM,
- smoke tests for Hill Climbing, BFS, Beam Search.

9 How to Run (Phase 1)

9.1 Unit Tests

```
python -m pytest -q
```

9.2 Run Optimization

Because imports use `src.*`, run scripts as modules:

```
python -m scripts.run_hill_climb --task qa --backend mock --budget 60
python -m scripts.run_search --task qa --backend mock --algo beam --budget 120 --m
```

9.3 Ollama Backend

Set model name via environment variable:

```
export OLLAMA_MODEL="llama3.1:8b"
python -m scripts.run_search --task summarization --backend ollama --algo bfs --bu
```

10 Conclusion and Phase 2 Preview

Phase 1 delivered a modular prompt optimization framework with classic search algorithms, deterministic metrics, trace logging for convergence plots, and a local LLM backend via Ollama. Phase 2 will extend experiments to at least 10 real-world inputs per task, perform empirical runtime/memory analysis, implement optional optimizers (simulated annealing, genetic algorithm), and produce comparative performance tables and plots aligned with theoretical complexity.